

DAY 11 — LANGGRAPH AGENT + TOOL CALLING + SQL/RAG ORCHESTRATION

✓ GOAL

Move from:

manual retrieval ✓

to:

AI engineering copilot ✓

..

that can:

- ✓ decide when to use RAG
 - ✓ decide when to use SQL
 - ✓ combine both
 - ✓ answer engineering questions intelligently
-

✓ DAY 11 ARCHITECTURE

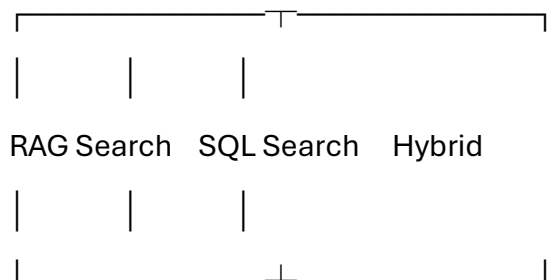
User Question

↓

LangGraph Agent ✓

↓

Tool Selection



↓

Context Assembly

↓

Qwen2.5



Final Engineering Answer

✅ WHAT DAY 11 ADDS

- ✅ LangGraph agent
 - ✅ tool calling
 - ✅ query routing
 - ✅ SQL orchestration
 - ✅ RAG orchestration
 - ✅ engineering copilot workflow
 - ✅ intelligent retrieval selection
-

✅ TEMPLATE STRUCTURE

app/

```
└─ agent/
|   └─ tools.py ✅
|   └─ router.py ✅
|   └─ graph.py ✅
|   └─ ask.py ✅
`
```

✅ STEP 1 — INSTALL

pip install langgraph langchain

✅ STEP 2 — CREATE TOOLS

📄 app/agent/tools.py

```
from app.query.hybrid_search import (
    hybrid_search
)

from app.query.structured_search import (
    structured_search
)
```

```
def rag_tool(query):

    results = hybrid_search(query)

    return {
        "tool": "rag",
        "results": results
    }
```

```
def sql_tool(query):

    results = structured_search(
        keyword=query
    )

    return {
        "tool": "sql",
        "results": results
    }
```

app/agent/router.py

```
def route_query(query):
```

```
    query = query.lower()
```

```
    sql_keywords = [
```

```
        "count",
```

```
        "maximum",
```

```
        "minimum",
```

```
        "average",
```

```
        "compare",
```

```
        "page"
```

```
    ]
```

```
    for k in sql_keywords:
```

```
        if k in query:
```

```
            return "sql"
```

```
    return "rag"
```

STEP 4 — LANGGRAPH WORKFLOW

app/agent/graph.py

```
from app.agent.router import (
```

```
    route_query
```

```
)
```

```
from app.agent.tools import (
    rag_tool,
    sql_tool
)
```

```
def run_agent(query):
```

```
    route = route_query(query)
```

```
    if route == "sql":
```

```
        return sql_tool(query)
```

```
    return rag_tool(query)
```

✅ STEP 5 — FINAL AGENT

 **app/agent/ask.py**

```
import ollama
```

```
from app.agent.graph import (
    run_agent
)
```

```
def ask_question(query):
```

```
    tool_output = run_agent(query)
```

```
    context = ""
```

```
#  RAG context
```

```
if tool_output["tool"] == "rag":
```

```
    for r in tool_output["results"]:
```

```
        context += (
            r["document"] + "\n"
        )
```

```
#  SQL context
```

```
else:
```

```
    for r in tool_output["results"]:
```

```
        context += (
            str(r) + "\n"
        )
```

```
prompt = f"""
```

You are an automotive engineering assistant.

Use ONLY the provided context.

Context:

{context}

Question:

{query}

```
"""
```

```
response = ollama.chat(

    model="qwen2.5",

    messages=[

        {

            "role": "user",

            "content": prompt

        }

    ]

)

return response["message"]["content"]
```

✅ STEP 6 — TEST AGENT

 **main.py**

```
from app.agent.ask import (

    ask_question

)
```

```
while True:
```

```
    query = input("\nAsk: ")
```

```
    if query.lower() == "exit":
```

```
        break
```

```
response = ask_question(query)
```

```
print("\n=== RESPONSE ===\n")
```

```
print(response)
```

✅ EXAMPLE FLOW

Query 1

What is front pad maximum temperature?

Agent decides:

✅ RAG search

because semantic retrieval better.

Query 2

Count all temperature rows

Agent decides:

✅ SQL search

because structured analytics better.

✅ WHY DAY 11 IS BIG

Before:

manual pipelines

Now:

AI decides retrieval strategy ✅

✓ CURRENT SYSTEM

PDF

↓

Extraction

↓

Normalization

↓

Filtering

↓

ChromaDB ✓

↓

DuckDB ✓

↓

LangGraph Agent ✓

↓

Qwen2.5 ✓

✓ YOU NOW HAVE

✓ local engineering copilot

✓ hybrid retrieval agent

✓ semantic + SQL orchestration

✓ tool-calling workflow

✓ enterprise-style RAG architecture

✓ YOU ARE NOW HERE

✓ DAY 11 COMPLETE

Next:

DAY 12

- conversational memory
- multi-step reasoning
- engineering workflows
- report summarization
- recommendation engine